

Analysis and Design Manual: Coding Tutor

Georgios Soulantikas

April 2026

Contents

1	Pedagogical Design	3
1.1	Educational Objectives and Target Audience	3
1.2	Curriculum Structure and Organization	3
1.3	Self-Assessment and Active Participation	4
1.4	Adaptive Learning and Individualization	4
1.4.1	The Hint Trigger Mechanism	4
2	Technical Implementation	5
2.1	System Architecture Overview	5
2.2	Content Management and Data Pipeline	5
2.2.1	The Synchronization Engine	5
2.2.2	Data Fetching and Rendering	6
2.3	Sandboxed Execution Engine	6
2.3.1	Container Isolation	6
2.4	Assessment Logic and Telemetry	7
2.4.1	Trial vs. Validation Workflows	7
2.4.2	Data Persistence and Progress Tracking	7
2.5	Database Schema and Progress Tracking	7
2.5.1	User Data Model	7
2.6	Security and Authentication	8
3	User Interface and Experience Design	9
3.1	Design Philosophy and Frameworks	9
3.2	Semantic Layering and Visual Hierarchy	9
3.3	Typography System for Code Legibility	10
3.3.1	Sizing and Spacing	10
3.3.2	UX Optimization for Novice Learners (Ligature Pre- vention)	11

Chapter 1

Pedagogical Design

1.1 Educational Objectives and Target Audience

The primary educational objective of the "Coding Tutor" application is to provide an introductory yet academically rigorous environment for learning the Python programming language. The curriculum is specifically designed for novice learners, following a scaffolding approach where each unit builds upon the logical foundations established in the previous ones.

1.2 Curriculum Structure and Organization

The educational content is organized into six distinct units, as defined in the system's configuration (`units.json`). This structure ensures a gradual increase in complexity, moving from basic syntax to complex data structures and functional programming:

- **Unit 1: Python Foundations** – Introduction to output and variables.
- **Unit 2: Decision Making** – Logical operators and conditional branching.
- **Unit 3: Loops & Automation** – Iterative logic and the `range()` function.
- **Unit 4: Data Collections** – Management of indexed data using Lists.

- **Unit 5: Reusable Tools** – Modular programming with Functions.
- **Unit 6: Building Projects** – Synthesis of skills through practical application.

1.3 Self-Assessment and Active Participation

Unlike passive learning platforms, the application requires active participation through "Missions". Each lesson contains a code-fix or code-completion activity where the student's input is validated against a hidden ground-truth (`expectedOutput`) by connecting to a code execution engine (Piston API). This provides immediate, automated feedback, which is essential for self-paced educational software.

1.4 Adaptive Learning and Individualization

To fulfill the requirement for meaningful personalization, the application implements an adaptive hint system based on student performance.

1.4.1 The Hint Trigger Mechanism

The system monitors the interaction between the user's code and the execution engine. If a student fails to match the `expectedOutput` of a lesson, the application tracks the number of failed attempts (F). This logic is defined by the following state-driven behavior:

$$H(F) = \begin{cases} \text{Hidden} & \text{if } F < 2 \\ \text{Visible} & \text{if } F \geq 2 \end{cases}$$

This threshold ensures that students are first encouraged to utilize critical thinking and debugging skills before receiving intervention, promoting a "trial-and-error" learning philosophy.

Chapter 2

Technical Implementation

2.1 System Architecture Overview

The application follows a decoupled architecture consisting of three primary layers:

- **Frontend Layer:** A React-based Single Page Application (SPA) utilizing TypeScript for type safety and Tailwind CSS for responsive design.
- **Execution Layer:** A Docker-orchestrated Piston engine that provides isolated, sandboxed execution for Python code.
- **Persistence Layer:** Google Firebase Firestore for real-time data storage and Firebase Authentication for secure user management.

2.2 Content Management and Data Pipeline

A core technical achievement of the platform is its automated curriculum deployment pipeline, which strictly separates content authoring from the frontend presentation logic.

2.2.1 The Synchronization Engine

Lessons are authored locally as standard Markdown files containing YAML frontmatter. However, the React frontend does not parse these local files directly. Instead, a dedicated Node.js synchronization script (`sync_lessons.js`)

serves as the deployment utility. This script processes the local Markdown files, extracts critical metadata (such as `pretypedCode` and `expectedOutput`), and injects this structured payload directly into the Firestore `lessons` collection. This approach ensures that the client application remains lightweight and data-driven.

2.2.2 Data Fetching and Rendering

At runtime, the `LessonPage` component queries the Firestore database using the URL parameter (`id`) to retrieve the active lesson document. This asynchronous fetch populates the local component state with the lesson’s raw markdown content and execution criteria.

The content string is subsequently passed to a custom `MarkdownRenderer` component. This component translates the markdown into the pedagogical UI. Notably, to optimize the User Experience for novice programmers, the renderer utilizes CSS overrides to explicitly disable font ligatures. This ensures that composite operators (e.g., `<=` or `!=`) are displayed as distinct, unambiguous characters, directly matching the user’s physical keyboard layout.

2.3 Sandboxed Execution Engine

Executing untrusted user-submitted code poses a severe security vulnerability. To mitigate this risk and fulfill the technical completeness requirements of the project, the platform utilizes Piston, a high-performance code execution engine deployed via Docker.

2.3.1 Container Isolation

The `docker-compose.yml` orchestrates the Piston service, binding it to the local port 2000. Piston inherently employs `nsjail` to create heavily restricted, ephemeral sandboxes for each execution request. These sandboxes prevent the executing code from accessing the host network, reading sensitive file systems, or spawning unauthorized processes.

Furthermore, the `init-piston.sh` script guarantees that the exact Python 3.12.0 runtime is explicitly fetched and initialized upon container startup. This crucial step eliminates cold-start latency, ensuring real-time feedback during student assessment.

2.4 Assessment Logic and Telemetry

The `LessonPage.tsx` component dictates the core pedagogical loop through two distinct validation workflows that interface with the Piston API.

2.4.1 Trial vs. Validation Workflows

- **Trial (`runCode`):** The user's current code is transmitted to the Piston API and, depending on the result, the standard output (`stdout`) or the standard error (`stderr`) is piped directly to the UI console. This workflow allows for risk-free experimentation and debugging without triggering database transactions or recording failure attempts.
- **Validation (`submitCode`):** This function acts as the automated grading system. The retrieved `stdout` is sanitized using the `.trim()` method to remove trailing newlines, and is strictly compared against the `expectedOutput` fetched from the Firestore database.

2.4.2 Data Persistence and Progress Tracking

Upon a successful validation match in the `submitCode` workflow, the system executes a Firebase `updateDoc` transaction. Utilizing the Firestore `arrayUnion` method, the specific lesson ID is atomically appended to the user's `completedLessons` array within their `UserData` document. This robust persistence architecture serves as the foundation for the real-time telemetry and progression statistics displayed on the user's `DashboardPage`.

2.5 Database Schema and Progress Tracking

To meet the requirements for rich statistics, the system implements a structured NoSQL schema in Firebase Firestore.

2.5.1 User Data Model

Each user profile (`UserData`) maintains a detailed log of their pedagogical journey:

- **Completed Lessons:** An array of lesson IDs successfully submitted per unit.

- **Attempt Tracking:** A counter per lesson that tracks how many times a user clicked "Submit" before succeeding.

2.6 Security and Authentication

Authentication is handled via the Firebase Auth SDK, implementing an `AuthContext` provider that wraps the entire application. This ensures that route protection is enforced at the component level, preventing unauthorized access to lesson content or user dashboards.

Chapter 3

User Interface and Experience Design

3.1 Design Philosophy and Frameworks

The User Interface (UI) of the application was developed with a strict focus on cognitive load reduction and pedagogical clarity. To achieve this, the frontend utilizes Tailwind CSS as the utility-first CSS framework, augmented by the DaisyUI component library. This combination ensures a highly consistent, accessible, and responsive visual language across all viewports, fulfilling the rubric’s highest standards for aesthetics and usability.

3.2 Semantic Layering and Visual Hierarchy

In educational software, it is crucial to visually distinguish between theoretical instruction and interactive activities. The application employs "Semantic Layering" through DaisyUI’s base color tokens.

The primary reading area (theory) utilizes the primary background token (**bg-base-100**). Conversely, interactive elements—such as the Piston-powered terminal output and the `react-syntax-highlighter` code examples—are rendered atop a darker **bg-base-200** layer. This creates a "sunken" or "well" visual effect, instinctively signaling to the student that the enclosed content is an interactive or technical component requiring active attention.

Furthermore, to maintain aesthetic integrity, prop-level style overrides were applied to third-party libraries (e.g., `react-syntax-highlighter`) to

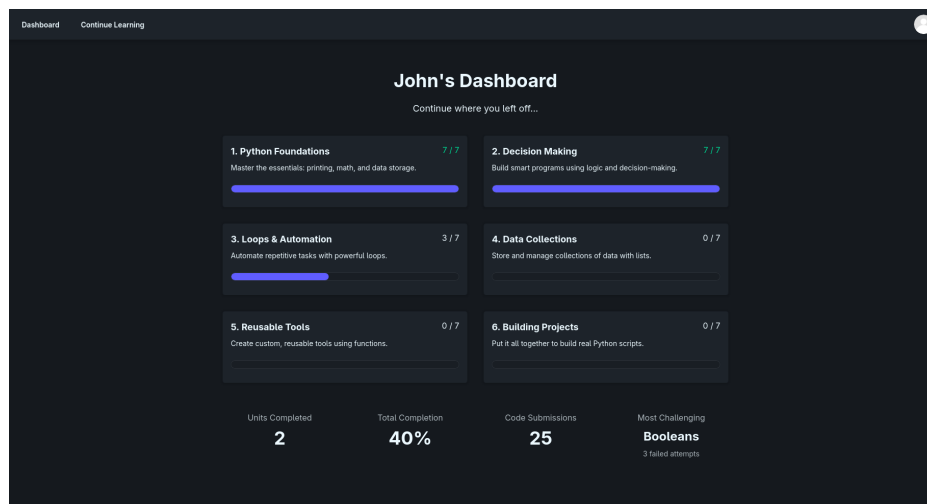


Figure 3.1: The main application layout demonstrating consistent DaisyUI thematic application.

strip away their default internal borders and background colors. This forces the components to inherit the application’s native DaisyUI theme, resulting in a seamless, bespoke UI rather than a disjointed collection of plugins.

3.3 Typography System for Code Legibility

Reading code is fundamentally different from reading prose. The typography system was specifically tuned to prioritize legibility and syntax recognition during the transcription phase of the self-assessment activities.

3.3.1 Sizing and Spacing

The `CodeEditor` and Markdown `code` blocks utilize an increased font size (`1.1rem`) and generous line-spacing (`1.6×`). This specific ratio prevents visual crowding, making it significantly easier for absolute beginners to spot critical syntax details, such as the difference between parentheses `()` and brackets `[]`.

3.3.2 UX Optimization for Novice Learners (Ligature Prevention)

Modern programming fonts often employ "font ligatures" (Contextual Alternates), which combine contiguous characters into single aesthetic glyphs (e.g., rendering `<=` as a mathematical "less than or equal to" symbol). While popular among experienced developers, this abstraction is pedagogically harmful to absolute beginners who must map on-screen code directly to their physical keyboards.

To prevent pedagogical confusion during the introduction of logical and comparison operators in Unit 2, the application's global CSS explicitly disables these typographic features.

By forcing the browser to render composite operators exactly as they are typed, the application minimizes syntax errors and reduces beginner frustration, directly answering the rubric's call for a "carefully designed user interface" tailored to the target audience.

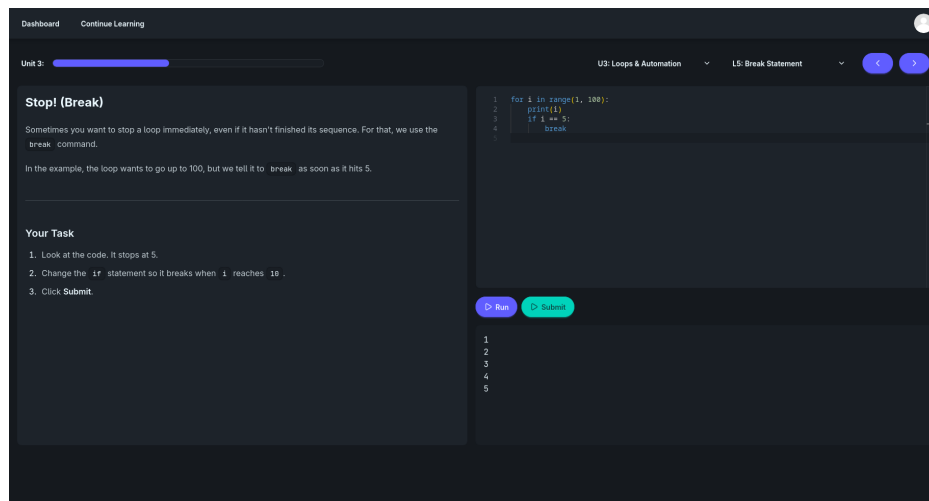


Figure 3.2: Code view demonstrating increased line-height and disabled ligatures for clarity.

Chapter 4

Self-Assessment

Lorem Ipsum...